AN INDEPENDENT MACHINE-CHECKED PROOF OF DETERMINISTIC FACTORING IN TIME $2^{O(n/\sqrt{\log n})}$, VIA THE NEW #SAT ALGORITHMS

VAIBHAV TULSYAN

Abstract. Before 2026, every proven deterministic factoring bound had a fixed positive exponent, the best being $N^{1/5+o(1)}$. In October 2026 Rudich and Rousseau used the new #SAT counting engines to prove in Lean that one Turing machine computes the least prime factor of an n-bit integer in time $2^{o(n)}$. They conjectured the rate $2^{O(n/\sqrt{\log n})}$ as a Lean proposition, `SqrtLogLogConjecture`. Shunia proved this rate on paper, conditional on two unrefereed preprints, and a public repository (`1337isnot1337/integer-factoring`, 30 October 2026) proved it in Lean using Rudich and Rousseau's multiplication network. However, we know of only this one machine-checked proof. We present a second one, built independently along a different route. Our multiplication network is a two-dimensional Schönhage–Strassen transform over $\mathbb{Z}/(2^J+1)$ with power-of-two parameters and no twiddle factors. Its value is the number of divisors of N with a given binary prefix, its shared bits total at most $2^{15}(n+2)$, and each of its g nodes holds $O(n/g+n^{3/4})$ bits, so Anthropic's counting theorem evaluates it in $2^{O(n/\sqrt{\log n})}$ steps. We prove in Lean 4 (`minFac_TM_rate`; axioms `propext`, `Classical.choice`, `Quot.sound`) that a finite-alphabet Mathlib Turing machine outputs $\mathrm{minFac}(N)$ within $\lfloor C \cdot 2^{Cn/\sqrt{\log_2(n+2)}} \rfloor$ steps for every N. From this we derive Rudich and Rousseau's `SqrtLogLogConjecture`, in their definitions copied verbatim and with constant $2C$, and their headline theorem. We claim no priority for the rate or for a formal proof, and we have not rebuilt the other development. The constants are astronomically large, the machine uses exponential space, and the full factorisation is not formalised. The proof rests on Anthropic's Lean development of its claimed refutation of the Exponential Time Hypothesis, which is under review.

1. Introduction

Let $\mathrm{minFac}(N)$ be the least prime factor of N. Computing it deterministically is a classical problem, and before 2026 every proven deterministic bound had a fixed positive exponent: $N^{1/4+o(1)}$ by Pollard [13] and Strassen [18], $N^{2/9+o(1)}$ by Hittmeir [7], and $O(N^{1/5}\log^{16/5} N)$ by Harvey [5], improved by a factor $(\log\log N)^{3/5}$ by Harvey and Hittmeir [6]. Randomised algorithms do far better; Lenstra and Pomerance prove the expected time $\exp((1+o(1))\sqrt{\log N \log\log N})$ [10].

In October 2026 OpenAI [12] and Anthropic [2] (below [O] and [A]) each claimed a deterministic algorithm that counts the satisfying assignments of a k-CNF in subexponential time, refuting the Exponential Time Hypothesis. Both claims come with Lean 4 developments and are still under review. Within days the counting engines were applied to factoring. The timeline, taken from the sources and their public records, is as follows.

- *29 October.* Rudich and Rousseau [14] (below RR) published a Lean proof that one deterministic Turing machine computes minFac within $C_k 2^{n/k}$ steps for every k, that is, in time $N^{o(1)}$. They state the rate $2^{O(n/\sqrt{\log n})}$ only as a conjecture, the Lean proposition `SqrtLogLogConjecture`.
- *30 October.* A written proof of that rate was merged into the public repository `1337isnot1337/integer-factoring` [8] (pull request #19). Its Lean formalisation (pull request #20, merged the same day) proves RR's exact proposition, with endpoint `IntegerFactoring.HalfAppendix.sqrtLogLog_factoring`. It composes RR's multiplication network with the clique evaluator of [A]'s Appendix B. The same day, a reviewer for the Big O Speedrun leaderboard reported that a rebuild of the repository at commit `e91442d3` was accepted by the Lean kernel and by a second kernel, nanoda [3]. The repository credits "substantial OpenAI assistant support". The fork `orda-aska/integer-factoring` carries the same proof.

Date: November 2026.

2020 *Mathematics Subject Classification.* 11Y05, 68Q25, 11Y16, 03B35.

Key words and phrases. Integer factorisation, deterministic algorithms, exact counting, Schönhage–Strassen multiplication, Exponential Time Hypothesis, Lean 4, formal verification.

- *31 October (published 2 November).* Shunia [17] gave a one-page proof of the rate on paper, with polynomial space and the full factorisation. It composes OpenAI's Boolean tensor-contraction theorem [12, Thm. D.1] with the multiplication network of Akshima et al. [1], and is conditional on these two unrefereed preprints.

However, we know of only one machine-checked proof of the rate, and its multiplication network is RR's. The developments involved are long and machine-generated: RR report about 35,000 lines for their proof, and about 152,000 for [A]'s. A second proof that shares as little as possible with the first is a cheap and useful check.

1.1. **The prefix-count problem.** We consider the *prefix-count problem*: given $N < 2^n$ and a binary word π of length at most n, count the divisors of N whose n-bit binary word starts with π. RR observed that n prefix counts find $\mathrm{minFac}(N)$, one bit at a time. So the question is how cheaply one prefix count can be posed to a counting engine.

Example 1.1 (Running example)**.** Take $N = 91 = 7 \cdot 13$, so $n = 7$. The divisors $1, 7, 13, 91$ have the words 0000001, 0000111, 0001101, 1011011. Let $D(\pi)$ count the pairs (p, q) with $pq = N$ whose factor p has the prefix π, and let $\mathrm{cnt}'(\pi) = D(\pi) - [\pi \text{ is a prefix of } 0000001]$, which removes the trivial divisor. The search keeps a prefix π, and extends it by 0 if $\mathrm{cnt}'(\pi 0) > 0$ and by 1 otherwise:

query $\pi 0$	0	00	000	0000	00000	000010	0000110
$D(\pi 0)$	3	3	3	2	1	0	0
$\mathrm{cnt}'(\pi 0)$	2	2	2	1	0	0	0
new π	0	00	000	0000	00001	000011	0000111

The result is $0000111 = 7 = \mathrm{minFac}(91)$. The fifth query shows why the correction is needed: the only divisor with prefix 00000 is 1.

1.2. **The naive encodings fail; the principle.** At first, one might pose the count as a #SAT instance: Tseitin's transformation of a multiplication circuit. However, no multiplication circuit of linear size is known (the best known have at least about $n \log n$ gates), and [A]'s clause bound $2^{O(m/\sqrt{\log m})}$ then gives only $2^{O(n\sqrt{\log n})}$, which is worse than brute force. [A]'s network model is more generous than clauses (Section 2): a node may hold many bits and check any polynomial-time predicate on them, and the engine charges roughly $2^{e \cdot U}$, where U is the number of bits the nodes share and $e = O(1/\sqrt{\log n})$. A schoolbook multiplication network still fails, because spreading its ℓ^2 partial products over g nodes makes the nodes share $\Omega(n\sqrt{g})$ bits (Section 4.4). Instead, we rely on a different principle: *the engine charges for shared bits, not for work inside a node, so we want a multiplication network with a constant number of stages, each of which cuts only $O(n)$ bits.* A two-dimensional Fourier transform has this shape. Each stage is a family of independent row or column transforms of $\Theta(n^{1/4})$ words of $\Theta(n^{1/2})$ bits, and one node checks a whole transform.

1.3. **What we prove.** In this paper we build such a network, prove its value and its parameters in Lean, and compose it with [A]'s counting theorem through our own interface to [A]'s development [19].

Theorem 1.2 (`EthFollowup.Factoring.minFac_TM_rate`; machine-checked)**.** *There are a Turing machine M in Mathlib's `TM2` model, with finite alphabets on all stacks, and a constant $C \in \mathbb{N}$ with the following property. For every $N \in \mathbb{N}$, started on the binary word `encodeNat` N of length n, the machine halts within*

$$\left\lfloor C \cdot 2^{\,Cn/\sqrt{\log_2(n+2)}} \right\rfloor$$

steps with output `encodeNat` $(\mathrm{minFac}\, N)$.

Here n is the bit length of N (zero for $N = 0$), and $\mathrm{minFac}(0) = 2$, $\mathrm{minFac}(1) = 1$, as in Mathlib. The two corollaries are RR's statements, proved in RR's own definitions (Section 6).

Corollary 1.3 (`EthFollowup.Factoring.sqrtLogLog`; machine-checked)**.** *RR's proposition `RR.Factoring.SqrtLogLogConjecture` holds: some finite-alphabet `FactoringMachine` factors every N within $C' \cdot 2^{\lfloor C' n/(\lfloor\sqrt{\lfloor \log_2 n \rfloor}\rfloor+1)\rfloor}$ steps, with $C' = 2C$ for the C of Theorem 1.2.*

Corollary 1.4 (`RR.Factoring.subexponentialRate`; machine-checked)**.**
One finite-alphabet `FactoringMachine` factors every N within $C_k \cdot 2^{\lfloor n/k \rfloor}$ steps for every $k \geq 1$, with $C_k = C \cdot 2^{C \cdot 2^{C^2 k^2}+1}$.

Corollary 1.4 is RR's headline theorem, which they already proved in Lean; we obtain it only as a consequence. As RR and Shunia note, running the machine again on $N/\operatorname{minFac}(N)$ gives the full factorisation within n rounds, hence in time $2^{O(n/\sqrt{\log n})}$. We have not formalised that loop, so we state it informally only.

1.4. **Contributions.** Our contributions are:

- We present a second machine-checked proof of RR's `SqrtLogLogConjecture`, in their definitions copied verbatim, that shares no multiplication-network code with the first and uses a different interface to [A]'s development (Sections 6 to 8).
- We construct an explicit parsimonious multiplication network, a two-dimensional Schönhage–Strassen transform over $\mathbb{Z}/(2^J+1)$ with power-of-two parameters and no twiddle factors, and prove in Lean that its value is the number of pairs (p,q) with $pq=N$ and a prescribed prefix of p, that its shared bits total at most $2^{15}(n+2)$, and that each of its g nodes holds at most $2^{16}((n+2)/g+d(n))$ bits, with $d(n)=O(n^{3/4})$ (Section 4).
- We do the exponent bookkeeping on [A]'s counted recursion: the level K satisfies $K^{40}\le K_0^{40}(n+2)$, and we explain why the exponent must be 40 and not the 30 of our earlier work (Section 5).
- We list the definitions a reader must trust (Section 6) and the differences from the first formal proof (Section 8).

The network is the textbook separable two-dimensional transform, and we do not claim it as new; what we add is its Lean proof and the parameter bounds that the engine needs.

1.5. **Status of each claim.** We separate four kinds of claim. *Machine-checked in Lean*: Theorem 1.2 and Corollaries 1.3 and 1.4, as `EthFollowup.Factoring.minFac_TM_rate`, `EthFollowup.Factoring.sqrtLogLog` and `RR.Factoring.subexponentialRate`, with the axioms `propext`, `Classical.choice` and `Quot.sound` only, and every lemma of ours whose Lean name we give. *Proved on paper only*: the full factorisation, and the explanations of Section 4.4 (why schoolbook and clause encodings fail), which are not used by the proof. *Assumed*: nothing beyond the three axioms; [A]'s results enter as Lean theorems of its development, which the kernel checks with ours (Section 7). *Checked numerically*: the running example and the parameter values of Example 4.1, by a short exact-arithmetic computation that is not part of the proof. Although this note does not improve any bound, we hope that a second, independent proof, and a network short enough to check on paper, are useful to those reviewing the engines.

Outline. Section 2 recalls [A]'s network model and counting theorem. Section 3 gives the prefix search. Section 4 builds the network and proves its value. Section 5 does the exponent arithmetic. Sections 6 and 7 describe the statement and the Lean development. Sections 8 and 9 discuss related work and limitations.

2. The engine counts accepted labellings

In this section we recall the part of [A]'s network model that we use, and the one theorem of [A] that we compose with. We use [A]'s network model in the form of its Lean development, at the commit pinned in our repository (`anthropics/formal-math`, folder `eth`, commit `16a5ce4`, Lean v4.33.1, Mathlib `0df444a`). We state only what we use.

Definition 2.1 ([A], Definition 2, abridged)**.** A *network on* $[g]$ *with parameters* P,B,τ has, for every node a, a finite set S_a of settings encoded as bit strings of length at most P and a weight $W_a\colon S_a\to\mathbb{Z}$; for every pair $\{a,b\}$ an interaction $\Psi_{a,b}$ that factors through $c_{a,b}$ shared bits; with $\sum_{b\neq a}c_{a,b}\le P$, all values below 2^B in absolute value, and every value computable with at most τ operations. Its value is $Z=\sum_z\prod_a W_a(z_a)\prod_{\{a,b\}}\Psi_{a,b}(z_a,z_b)$, and its *total* is $U=\sum_{a\neq b}c_{a,b}$.

We use the special case of a *label system*: a set of items (our wires), each placed on a pair of nodes and carrying a label of W bits; both nodes of the pair see the label, Ψ is equality of the two copies, and W_a is a 0/1 table that reads the labels at a. Then the value is the number of labellings that every table accepts. In Lean this is the `LabelSystem` of our earlier work [19] on [A]'s `Net` format.

Imported Theorem 2.2 ([A], Lemma 8(i) and Theorem A(b), with the pieces of Theorem B)**.** *Let* $K\ge 2$, $g=3K^8$, *and let* $K=K_\ell$ *be a level of* [A]*'s schedule, with* $e_\ell\le(3x_0+6)/\sqrt{\log_2 K_\ell}$ *for an absolute* x_0. *A*

network on $[g]$ with parameters P, B, τ and total U can be evaluated within

$$\Big(K^g + 2^{e_\ell K^2 L}\big(K(L+B')\big)^{C_0 K^2} \cdot 2^{2gP/K} \cdot \tau\Big) \cdot \big(g(P+B)\big)^{C_1} \tag{2.1}$$

operations, where $L = \lceil (U + gP/K)/K^2 \rceil$, $B' = \lceil 2gP/K \rceil + g^2 B$, and C_0, C_1 are absolute.

In Lean, (2.1) is `ExplicitCost.groupingExplicit` (Lemma 8, case (i), which groups the g nodes into K groups by trying all K^g groupings) together with the clique machine of `CountedRoute.onTheCountedRecursion` (Theorem A(b) on the pieces of Theorem B). The procedure queries the tables W_a and the factorisations of Ψ; we answer the queries with polynomial-time procedures, so no table is ever written out. [A]'s host lemma `CountKSat.exists_tmW` turns the resulting step-by-step machine into a Mathlib Turing machine with a polynomial overhead $q(T+1)(|x|+S+1)^q$ in its time T and state size S. We used the same route in [19] (module `ExactHC.Theorem2`) and reuse its level selection.

3. Prefix counts find the least prime factor

In this section we show that n prefix counts find $\mathrm{minFac}(N)$. This part is pure Mathlib (module `Search`). Write $\mathrm{word}_n(d)$ for the big-endian n-bit word of d and $\mathrm{val}(\pi)$ for the value of a word π.

Definition 3.1. For $N, n' \ge n$ and a word π of length $t \le n$, let

$$D(N, n', n, \pi) = \#\big\{(p,q) \in [0, 2^{n'})^2 : pq = N,\ \lfloor p/2^{n-t} \rfloor = \mathrm{val}(\pi)\big\},$$

and $\mathrm{cnt}'(\pi) = D(N, n', n, \pi) - [\lfloor 1/2^{n-t} \rfloor = \mathrm{val}(\pi)]$.

Lemma 3.2 (`D_eq_card_divisors`, `cnt'_eq`)**.** *If $1 \le N < 2^n$ and $n \le n'$, then $D(N, n', n, \pi)$ is the number of divisors $d \mid N$ whose word $\mathrm{word}_n(d)$ has prefix π, and $\mathrm{cnt}'(\pi)$ is the number of such divisors $d \ne 1$.*

Proof. A pair with $pq = N \ge 1$ has $p, q \le N < 2^n \le 2^{n'}$, so the box imposes nothing, and p determines q. The prefix condition is $\lfloor p/2^{n-t} \rfloor = \mathrm{val}(\pi)$ (`prefix_word_iff`). □

Proposition 3.3 (`searchWith_eq_minFac`)**.** *Let $2 \le N < 2^n$, and let the oracle c agree in positivity with cnt' on every query $\pi 0$ with $|\pi| < n$. Starting from the empty word and replacing π by $\pi 0$ if $c(\pi 0) > 0$ and by $\pi 1$ otherwise, after n rounds the word has value $\mathrm{minFac}(N)$.*

Proof. The invariant is $\mathrm{val}(\pi) = \lfloor \mathrm{minFac}(N)/2^{n-|\pi|} \rfloor$. If $\mathrm{cnt}'(\pi 0) > 0$, some divisor $d \ne 1$ has prefix $\pi 0$; since $d \ge \mathrm{minFac}(N)$ and both have prefix π, the next bit of $\mathrm{minFac}(N)$ is 0. Otherwise no divisor $d \ne 1$ has prefix $\pi 0$, but $\mathrm{minFac}(N)$ has prefix π, so its next bit is 1. Numeric order is lexicographic order on words of equal length (`ofWord_lt_iff_lex`). □

The machine handles $N \in \{0, 1\}$ directly. For $N \ge 2$, each round asks one count $D(N, n', n, \pi 0)$, which the network of Section 4 computes, and subtracts the correction bit, which is 1 exactly when π is all zeros and $|\pi| + 1 < n$ (`corrB` in `Rate`).

4. A two-dimensional transform network counts the divisors

In this section we fix the parameters, describe the stages of the transform, turn them into wires and checks, prove that the accepted labellings correspond to the counted pairs, and group the vertices into g clusters. All quantities are functions of n. Put $\nu = \lfloor \log_2(n+2) \rfloor$ and

$$m = \lfloor \nu/4 \rfloor + 1, \qquad L = 2^m, \qquad \ell_1 = 2^{m-1}, \qquad \ell = \ell_1^2, \qquad b = \lfloor n/\ell \rfloor + 1,$$
$$J = 2^{\lfloor \nu/2 \rfloor + 7}, \qquad M = 2^J + 1, \qquad \omega = 2^{2J/L} \in \mathbb{Z}/M, \qquad W = J + 2.$$

So $L = 2\ell_1$, all parameters are powers of two except b and W, and $n < n' := \ell b$. One checks (module `Params`) that $L \mid 2J$, $2^{2J} \equiv 1$, $\omega^{L/2} = 2^J = -1$, $L^{-1} = 2^{2J-m}$ in $\mathbb{Z}/M$, and

$$2b + 2m + 1 \le J, \qquad \ell \cdot 2^{2b} \le 2^J. \tag{4.1}$$

The second inequality says that a coefficient of the product of two limb arrays, a sum of at most ℓ products of b-bit limbs, is less than M, so its residue modulo M is its value.

Example 4.1 (Parameters)**.** For $n = 2^{20}$: $\nu = 20$, $m = 6$, $L = 64$, $\ell_1 = 32$, $\ell = 1024$, $b = 1025$, $J = 2^{17}$. There are 28,735 wires of $W = 131{,}074$ bits, about $3600\,n$ bits in all. A transform vertex reads $2LW \approx 2^{24}$ bits, which is more than n: at this size the network is far from useful, and $n^{3/4}$ beats n only for much larger n. For the running example $n = 7$, the parameters degenerate to one limb.

4.1. **Stages.** Write $p = \sum_{i_1<\ell_1,\, i_2<\ell_1} p_{i_1 i_2} 2^{b(i_1+\ell_1 i_2)}$ with b-bit limbs, and the same for q. This is a bivariate Kronecker substitution: p is the value at $X = 2^b$, $Y = 2^{b\ell_1}$ of a polynomial of degree $< \ell_1$ in each variable. The product pq is the value at the same point of the two-dimensional acyclic convolution $c_{a,u} = \sum_{i_1+j_1=a,\ i_2+j_2=u} p_{i_1 i_2} q_{j_1 j_2}$, with $a, u < L$. Because $L = 2\ell_1$, the acyclic convolution equals the cyclic one of size $L \times L$, which we compute with a separable two-dimensional discrete Fourier transform over $\mathbb{Z}/M$:

(i) *row transforms*: $A_{s,i_2} = \sum_{i_1} p_{i_1 i_2}\, \omega^{i_1 s}$ for $s < L$ (same for q);
(ii) *column transforms*: $\hat{p}_{s,t} = \sum_{i_2} A_{s,i_2}\, \omega^{i_2 t}$ for $t < L$;
(iii) *pointwise products*: $\hat{c}_{s,t} = \hat{p}_{s,t}\, \hat{q}_{s,t}$;
(iv) *inverse columns* and *inverse rows*, each with the factor $L^{-1} = 2^{2J-m}$ and the exponents $\omega^{(L-u)t}$, giving $c_{a,u}$ by the two-dimensional convolution theorem (`idft2_dft2_mul`);
(v) *carries*: position $k = a + \ell_1 u$ collects at most two coefficients ($a < 2\ell_1$), their sum γ_k, and the checks $N_k \le \gamma_k + r_k$, $(\gamma_k + r_k - N_k) \equiv 0 \pmod{2^b}$ and $r_{k+1} = (\gamma_k + r_k - N_k)/2^b$, with $r_0 = r_{\text{last}} = 0$ and N_k the k-th limb of N.

There are no twiddle factors. In the one-dimensional "four-step" version of the transform, as in Akshima et al. [1] and as RR describe theirs, the stages are coupled by twiddle factors; the two-dimensional form avoids them because the transform of a bivariate polynomial is separable. Every power ω^e is $2^{(2J/L)(e \bmod L)}$, a shift. The carry chain is correct by `chain_iff`: a valid chain exists if and only if $\sum_k \gamma_k 2^{bk} = N$, and its carries are then unique (`chain_unique`).

4.2. **Wires, vertices and checks.** Every intermediate value is a *wire* with a label in $[0, 2^W)$: the limbs of p and q (ℓ each), the row outputs ($2L\ell_1$), the column outputs, products, inverse columns and coefficients (L^2 each, five families), and the carries. Every vertex is a check on the wires it touches:

- a *cell* for each limb: the limb is below 2^b and, for p, its bits match the *pattern* of the prefix (bits of p above position $n - |\pi|$ fixed to π, bits at positions $\ge n$ fixed to 0; `patOK_iff`);
- a *row vertex* for each i_2 and a *column vertex* for each s: all its outputs are the canonical residues of the transform of its inputs; likewise for the inverse transforms;
- a *product vertex* for each (s, t), and a *carry vertex* for each position k.

A row or column vertex reads at most $2L$ wires, so at most $2LW \le d(n) := 2^{\lfloor 3\nu/4 \rfloor + 11}$ bits; $d(n)^4 \le 2^{44}(n+2)^3$, so $d(n) = O(n^{3/4})$. There are at most 30ℓ wires, carrying at most $2^{14}(n+2)$ bits in total (`numWires_mul_W_le`).

Theorem 4.2 (Parsimony; `mulNet_card`, `mulNet_value`)**.** *Let $N < 2^n$ and $|\pi| \le n$. The labellings of the wires that pass every check are in bijection with the pairs $(p, q) \in [0, 2^{n'})^2$ with $pq = N$ and $\lfloor p/2^{n-|\pi|} \rfloor = \mathrm{val}(\pi)$. So the network's value is $D(N, n', n, \pi)$.*

Proof. Forward (`run_all_iff`). Given (p, q), let $\mathrm{run}(p, q)$ label every wire with its canonical value. All transform and product checks pass by definition. By the convolution theorem, (4.1) and the Kronecker evaluation (`pipeline_eq_mul`), the coefficients sum to pq, so the carry checks pass if and only if $pq = N$ (`carry_checks_iff`); the cells pass if and only if p matches the pattern. *Backward* (`eq_run`). If a labelling passes every check, read p, q off the limb wires. Each stage's check determines its outputs from its inputs, so by induction over the stages every wire carries its canonical value; the carry labels are forced by `chain_unique`. So the labelling is $\mathrm{run}(p, q)$. The two maps are inverse, which gives the bijection. □

In the running example ($N = 91$, $\pi 0 = 00000$), the only accepted labelling is $\mathrm{run}(1, 91)$, and the value is 1, as in the table of Example 1.1.

4.3. **Clusters.** To obtain a network on $[g]$, we split the vertices of each stage into g contiguous blocks of nearly equal size, and let cluster a consist of the a-th block of every stage. A wire whose two end vertices lie in clusters $a \ne c$ sits on the pair $\{a, c\}$; a wire with both ends in cluster a sits on $\{a, a+1\}$ (or $\{0, a\}$ for the

last cluster), which costs nothing asymptotically. The table of node a is 1 if every check of its vertices passes and 0 otherwise, so $B = 1$.

Proposition 4.3 (`mulSys_U_le`, `card_inc_mul_W_le`, `mulSys_is`)**.** *For $g \geq 2$, the network has total $U \leq 2^{15}(n+2)$, and every node holds at most $P(n,g) = 2^{16}\big((n+2)/g + d(n)\big)$ bits; it is a network in the sense of Definition 2.1 with parameters $P(n,g)$ and $B = 1$.*

Sketch. U is at most twice the number of wire bits. A cluster holds at most $|\text{stage}|/g + 1$ vertices of each of the ten vertex kinds, each of degree at most the stage's degree, and every wire at a has an end in cluster a, $a-1$ or $g-1$. Summing gives $g \cdot \#\{\text{wires at } a\} \leq 3(64\ell + 15gL)$, and multiplying by W gives the bound. □

4.4. **Why schoolbook multiplication does not fit.** This subsection is an explanation on paper; the proof does not use it. Each stage of the transform pipeline cuts $\Theta(n)$ bits, and there are a constant number of stages, so any split into clusters shares $O(n)$ bits. This is what fails for schoolbook multiplication. If g balanced clusters share the ℓ^2 partial products $p_i q_j$, a cluster with ℓ^2/g of them needs at least $2\ell/\sqrt{g}$ limbs of the inputs, by the rectangle inequality, so the clusters together receive $\Omega(n\sqrt{g})$ bits. With $g = 3K^8$ a power of n and $e = \Theta(1/\sqrt{\log n})$, the term eU is then $n^{1+\Omega(1)}$. (Karatsuba-type recursions and residue number systems have the same problem, for the same reason.) A one-dimensional FFT with $\log n$ butterfly stages has a linear cut per stage but $\log n$ stages. The two-stage transform works because a node may check a whole row transform of $L = \Theta(n^{1/4})$ words of $\Theta(n^{1/2})$ bits at once.

5. The exponent is $O(n/\sqrt{\log n})$ at level $K^{40} \lesssim n$

In this section we choose the level of the engine and add up the terms of (2.1). The machine (`FR.facM` in `Rate`) reads n, chooses a level $K = K_\ell$, sets $g = 3K^8$, and runs the n rounds of Proposition 3.3; each round builds the context of the network for $(g, N, \pi 0)$ in polynomial time and runs the procedure of Imported Theorem 2.2 on it.

The level. Let $K = K_\ell$ be the largest level with $K_\ell \leq \max(\lfloor n^{1/40} \rfloor, K_0)$ (`kN40`). By [A]'s formula for $\log K_\ell$ (Theorem A(a)), $K_{\ell+1} \leq K_\ell^{10}$, so (`kfacts40`)

$$2 \leq K, \qquad K^{40} \leq K_0^{40}(n+2), \qquad n+2 \leq K^{40C_0+1}. \tag{5.1}$$

The last inequality gives $\log_2 K \geq \log_2(n+2)/(40C_0+1)$, so $e_\ell = O(1/\sqrt{\log n})$.

Why 40. The node bound forces $gP = O(n)$, and P is at least the largest vertex degree, $d(n) = \Theta(n^{3/4})$. So we need $g\,d(n) = O(n)$, that is $K^8 = O(n^{1/4})$, or $K = O(n^{1/32})$. Our earlier work [19] used $K^{30} \leq n$, which gives $g\,d(n) \approx n^{8/30+3/4} > n$. With $K^{40} \leq K_0^{40}(n+2)$ we get $g\,d(n) \leq c(n+2)^{1/5+3/4} = O(n)$ (`Kdg_le`).

The exponent. By Proposition 4.3, $U \leq 2^{15}(n+2)$ and $gP \leq 2^{16}((n+2)+g\,d(n)) = O(n)$. The polynomial-time procedures give τ and S polynomial in $n+g$ (the package bounds of Section 7). In (2.1), $K^2 L \leq U + gP/K + K^2$, so the logarithm of the time of one round is at most

$$g \log K \;+\; e_\ell\Big(U + \frac{gP}{K} + K^2\Big) \;+\; C_0 K^2 \log\big(K(L+B')\big) \;+\; \frac{2gP}{K} \;+\; O(\log n).$$

The second term is $O(n/\sqrt{\log n})$. The others are smaller: $g \log K = O(n^{1/5} \log n)$, $K^2 \log(\cdots) = O(n^{1/20} \log n)$, and $gP/K = O(n/K) = O(n^{1-1/(40C_0+1)})$. Multiplying by n rounds and by the polynomial overhead of `CountKSat.exists_tmW` changes only the constant. In Lean this is `FR.round_bound` (on top of the exponent calculus `T2.branch_bound` of [19]) and `FR.facM_runs`, and the result is `minFac_TM_of_pkg`: Theorem 1.2 for any *package* (Section 7).

From $\sqrt{\log_2(n+2)}$ to RR's denominator. RR write the rate as $C' \cdot 2^{\lfloor C' n/(s+1) \rfloor}$ with $s = \lfloor \sqrt{\lfloor \log_2 n \rfloor} \rfloor$. Since $s+1 \leq 2\sqrt{\log_2(n+2)}$ (`natSqrt_log_add_one_le`), we have $Cn/\sqrt{\log_2(n+2)} \leq \lfloor 2Cn/(s+1) \rfloor + 1$, and so $\lfloor C 2^{Cn/\sqrt{\log_2(n+2)}} \rfloor \leq 2C \cdot 2^{\lfloor 2Cn/(s+1) \rfloor}$ (`rateT_le_sqrtLogLog`). This is why the constant in Corollary 1.3 is $2C$. For Corollary 1.4, if $2^{C^2 k^2} \leq n+2$ then $C/\sqrt{\log_2(n+2)} \leq 1/k$; otherwise the exponent is at most $C \cdot 2^{C^2 k^2}$ (`rateT_le_subexp`).

6. What a reader must trust: RR's definitions

A reader who wants to trust Corollary 1.3 needs to check the following definitions, and nothing else of ours. They are RR's file `Challenge.lean` at commit `5f5ff8f3823fa8fef8ee02a6f29049db3c76a0ce` of their repository [15] (the commit registered as Palomar entry `PALOMAR-2026-10-28-000015`, version 1), file

SHA-256 `c3adce9c...ae12b`. We downloaded the file at that commit and at commit `c91cc3f`, the one pinned by [8]; the two copies are byte-identical. Our module `Statement` copies the definitions verbatim inside `namespace RR`, with two syntactic changes: the header uses `import` instead of `public import` (our file is not a Lean `module`), and the line `@[expose] public section` is dropped. We render the Unicode symbols as mathematics:

```
def factor : ℕ → ℕ := Nat.minFac
abbrev FactoringMachine := TM2ComputableInTime encodeNat encodeNat factor
def FactoringMachine.FiniteAlphabets (M : FactoringMachine) : Prop :=
  ∀ k, Finite (M.tm.Γ k)
def FactorsWithin (M : FactoringMachine) (T : ℕ → ℕ) : Prop :=
  M.FiniteAlphabets ∧ ∀ n, M.time n ≤ T n
def FactoringInTime (T : ℕ → ℕ) : Prop :=
  ∃ M : FactoringMachine, FactorsWithin M T
def SqrtLogLogConjecture : Prop :=
  ∃ C : ℕ, FactoringInTime
    (fun n => C * 2 ^ (C * n / (Nat.sqrt (Nat.log 2 n) + 1)))
```

RR's headline statement (their `subexponentialRate`) is

```
∃ M : FactoringMachine, ∀ k : ℕ, 1 ≤ k → ∃ C : ℕ,
  FactorsWithin M (fun n => C * 2 ^ (n / k))
```

In `Challenge.lean` its proof is a placeholder (RR prove it in a separate file); we do not copy that declaration, but state and prove the same proposition under the same name. Behind these definitions are Mathlib's `Turing.TM2ComputableInTime` (a `FinTM2` machine with input and output encodings and a time function), `Computability.encodeNat` (little-endian binary, the empty word for 0), `Nat.minFac`, `Nat.sqrt` and `Nat.log`, all from Mathlib `0df444a`. Division is natural-number division. Mathlib's `TM2ComputableInTime` asks only for a finite input alphabet; `FiniteAlphabets` asks it of every stack.

Theorem 1.2 is stated with [A]'s `GraphProblems.Outputs` $M\,x\,y\,T$, which says that M started on x halts within T steps with output y (a `TM2OutputsInTime` of Mathlib). The adaptor `toFactoringMachine` turns such a machine and time function into one of RR's `FactoringMachine`s, with the same machine and the same time function (both by `rfl`), and `outputs_of_factoringMachine` is the converse. So Theorem 1.2 and the statement "some `FactoringMachine` has time function $n \mapsto \lfloor C2^{Cn/\sqrt{\log_2(n+2)}} \rfloor$" are the same.

7. The Lean development

In this section we list the modules, describe the interface between the network and the machine, and state the trusted base. The development is in the public repository [20], directories `lean/EthFollowup/Factoring` (pure Mathlib; the files compile with Lean v4.33.1 and v4.34.1) and `lean-bridge-anthropic/bridge/EthFollowup/Factoring` (on top of [A]'s development). Table 1 lists the modules.

The package. The interface between the network and the machine is a structure `MulPackage` c_0 (in `Params`): a context type, one set of procedures, a reader that gets g in unary together with the input word and the prefix, the network for every $g \geq 2$, and the bounds value $= D$, $U \leq c_0(n+2)$, $P \leq c_0((n+2)/g + d(n))$, and B, τ, S at most $q(n+g+2)^q$ for one q. `minFac_TM_of_pkg` proves Theorem 1.2 from the proposition "there is c_0 with `Nonempty (MulPackage c0)`", and `MulPackage` proves this with $c_0 = 2^{16}$ (`mulPackage_nonempty`). One point needed care: the labels are W-bit words with $W \approx 2^7\sqrt{n}$, so the list of all code words that our earlier label procedures kept in the context has 2^W entries; since every W-bit word is a code word, the membership test is replaced by a length test (`labelProcsW` in `MulProcs`).

Trusted base and axioms. The theorems depend only on `propext`, `Classical.choice` and `Quot.sound`. A fresh `leanchecker` replay of the dependency closure of `EthFollowup.Factoring.Final`, the module holding the three final theorems, has passed. What must be trusted is the Lean kernel, the Mathlib definitions named in Section 6, and RR's definitions. [A]'s development, Mathlib's proofs and our modules are checked by the kernel; they are part of the checked closure, not of the trusted base. The proof does not import RR's development, only their statement.

Constants. The proved bounds are $\ell W \leq 2^8(n+2)$ and $L^2 W \leq 2^{10}(n+2)$; the degree bound is proved per stage (`degSt`); the constant in Corollary 1.3 is $2C$. None of the constants is optimised.

module	content	lines
`FermatRing`	orthogonality of ω with $\omega^{2^m} = -1$; the ring $\mathbb{Z}/(2^J+1)$	198
`DFT`	1D/2D transforms, inversion, cyclic convolution theorems	177
`Kronecker`	limb evaluation, coefficient bounds, value recovery	166
`Carry`	carry chains: existence iff sum, uniqueness	161
`Pipeline`	the $\mathbb{N}$-level pipeline computes the product	164
`Search`	prefix search, D, cnt$'$, minFac	411
`Params`	parameters, wires, vertices, checks, clusters, the package shape	1374
`FermatBridge`	`pipeline_eq_mul`, `carry_checks_iff`	114
`MulNetDefs`, `MulNetParsimony`, `MulNetCount`, `MulNet`	the label system, parsimony, U and P	731
`MulProcs`	procedures, context, layout, tables	730
`MulCtxPoly`	polynomial time of context and tables	441
`MulPackage`	the reader; `mulPackage_nonempty`	352
`Level40`	the level with exponent 40	154
`Rate`	the machine, round bound, `minFac_TM_of_pkg`	691
`Statement`	RR's definitions verbatim; the corollaries from the package	284
`Final`	the three final theorems	38
total		6186

TABLE 1. The modules, with line counts (`wc -l`). The first six are pure Mathlib. The development also reuses the bridge modules `LabelNetwork`, `LabelProcs`, `VertexBalanced`, `SearchMachine` and `Theorem2` of [19] (3693 lines), unchanged.

8. Related work

We organise related work by idea: deterministic factoring bounds, the counting engines applied to factoring, fast multiplication as a network, and formal proofs.

Deterministic factoring. The classical deterministic bounds are those of Pollard [13] and Strassen [18] ($N^{1/4+o(1)}$), Hittmeir [7] ($N^{2/9+o(1)}$), Harvey [5] ($O(N^{1/5}\log^{16/5} N)$) and Harvey and Hittmeir [6], who gain a factor $(\log\log N)^{3/5}$. Lenstra and Pomerance [10] give the rigorous randomised bound. These are concrete algorithms with modest constants. In contrast, our bound beats every fixed exponent only asymptotically, beats none of the randomised or heuristic methods, and rests on the counting engine of [A].

Counting engines applied to factoring. RR [14] introduced the reduction we use (prefix search by divisor counting) and the statement we prove; their own Lean proof gives $N^{o(1)}$ with no rate. Shunia [17] proves the rate on paper, with polynomial space and the full factorisation, conditional on OpenAI's Theorem D.1 and on Akshima et al.'s Theorem 3. Closest to our work is the Lean proof of [8]. It proves the same proposition in the same definitions, on the same commit of [A]'s development. As far as we can tell from its written proof and documentation, it differs from ours in two places. First, its multiplication network is RR's, taken from RR's development (RR's `mulNet_value`, at RR's commit `c91cc3f`). Ours is the network of Section 4, built and proved separately, so the two proofs share no multiplication-network code. Second, it runs [A]'s Appendix B clique evaluator (`AppRunningTime.thmRunningTime`) on explicitly written clique tables, with a top grouping found by search and an explicitly written unary pad of length 2^q that pays for the evaluator's running time. Ours runs Lemma 8's grouping procedure with the clique machine of the counted recursion, answers table queries with polynomial-time procedures, and writes neither tables nor a pad. We do not claim that either difference makes our proof stronger. Both proofs have the same audit surface: RR's definitions, Mathlib and the kernel. The closure of [8] includes RR's proof development and ours does not. Theirs was checked by a second kernel (nanoda) in the reviewer's rebuild, and ours only by Lean's. In contrast, we give a second machine-checked proof, with its own network written out in full in this note.

Fast multiplication as a network. The transform over $\mathbb{Z}/(2^J+1)$, with ω a power of 2 so that multiplications by ω are shifts, is Schönhage and Strassen's [16]. Splitting a transform into smaller ones goes back to Cooley and Tukey [4]. Akshima et al. [1] observed that one level of this transform, with large nodes and a four-step split, gives a network with $O(n)$ edges and degree $O(n^{3/4})$; RR's network is also a transform modulo a Fermat number split into two layers, followed by carries. In contrast, our network is two-dimensional (a bivariate

Kronecker substitution), so its two layers are independent row and column transforms with no twiddle factors, and all of its parameters are powers of two.

Formal proofs on the engines. RR and [8] build on the engines in Lean, as do our earlier notes [19], whose interface to [A]'s development (the label systems and the level selection of `ExactHC.Theorem2`) we reuse. We use Lean 4 [9] and Mathlib [11]. In contrast to our earlier notes, whose statements are in our own definitions, the statement here is a proposition stated by others before our proof existed.

9. Limitations

- *The result stands or falls with* [A]*'s development.* The proof rests on [A]'s Lean development of its claimed refutation of the Exponential Time Hypothesis, and the labs' claims are under review. Lean checks that development as part of our proof, and our statement uses only Mathlib and RR's definitions, so we do not need [A]'s formal statements to say what its printed theorems say. We do need the development, built from source at the pinned commit, to be a sound Lean development, and the toolchain to be sound.
- *Nothing here improves a bound.* The rate is Shunia's, and the first machine-checked proof is that of [8]. Our contribution is a second, independent proof and the explicit network.
- *The algorithm is galactic.* [A]'s levels start at $K_0 = 2^{x_0}$ with $x_0 \geq 18$, the level needs $K^{40} \lesssim n$, and the vertex degree $d(n)$ exceeds n until n is very large (Example 4.1). The algorithm is of no practical use.
- *The machine uses exponential space.* Shunia's route gives polynomial space, conditionally.
- *The full factorisation is not formalised.* It is stated informally only.
- *No human has checked the work.* The Lean code and this text were written with AI assistance. No human expert has yet checked that Section 6 lists everything a reader must trust, though the list is short. We have not rebuilt the development of [8].

Acknowledgements

This work was done with AI assistance (Claude Code, Anthropic): the construction, the Lean development, the literature and citation checks and the text were produced in sessions with Claude Code. The Lean theorems are checked by the Lean kernel; everything else awaits human expert review. Responsibility for any errors rests with the author. This note continues the project of [19].

References

[1] Akshima, R. Bacho, Y.-H. Huang, E. Kiltz, A. Nishaburi, S. Nooripoor, N. Tittelbach, and E. Wiedijk, *A subexponential time algorithm for elliptic curve discrete logarithm*, preprint, version 0.2, 29 October 2026, https://github.com/bochum-crypto/subexponential-ecdl.

[2] Claude (Anthropic), *A $2^{o(n)}$ time algorithm for 3-SAT*, preprint dated 26 October 2026. Paper and Lean formalisation in the repository https://github.com/anthropics/formal-math, folder `eth`; we use commit `16a5ce41304fcdaea4e0be7c132e1c361dc26da5`.

[3] GitHub user nmamano, writing for the Big O Speedrun reviewers, *Big O Speedrun: alpha = 1/2 listed as Claimed; a lake-packaged commit for a Lean rerun*, review comment on issue #28 of [8], 30 October 2026, https://github.com/1337isnot1337/integer-factoring/issues/28.

[4] J. W. Cooley and J. W. Tukey, *An algorithm for the machine calculation of complex Fourier series*, Math. Comp. **19** (1965), 297–301.

[5] D. Harvey, *An exponent one-fifth algorithm for deterministic integer factorisation*, Math. Comp. **90** (2021), no. 332, 2937–2950.

[6] D. Harvey and M. Hittmeir, *A log-log speedup for exponent one-fifth deterministic integer factorisation*, Math. Comp. **91** (2022), no. 335, 1367–1379.

[7] M. Hittmeir, *A time-space tradeoff for Lehman's deterministic integer factorization method*, Math. Comp. **90** (2021), no. 330, 1999–2010.

[8] 1337isnot1337 (GitHub user) and contributors, *integer-factoring: deterministic integer factoring bounds*, public repository https://github.com/1337isnot1337/integer-factoring. Written proof `research/half-rate-factoring.md` (pull request #19, merged 30 October 2026); Lean formalisation in `formal-appendix/` (pull request #20, merged 30 October 2026), endpoint `IntegerFactoring.HalfAppendix.sqrtLogLog_factoring`. Fork: https://github.com/orda-aska/integer-factoring.

[9] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, Automated Deduction – CADE 28, Lecture Notes in Comput. Sci., vol. 12699, Springer, 2021, pp. 625–635.

[10] H. W. Lenstra, Jr. and C. Pomerance, *A rigorous time bound for factoring integers*, J. Amer. Math. Soc. **5** (1992), no. 3, 483–516.

[11] The mathlib Community, *The Lean mathematical library*, Proc. 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), 2020, pp. 367–381.
[12] OpenAI, *The Exponential Time Hypothesis is false*, preprint dated 17 October 2026, revised 25 October 2026. Paper and Lean formalisation in the repository https://github.com/openai/math.
[13] J. M. Pollard, *Theorems on factorization and primality testing*, Proc. Cambridge Philos. Soc. **76** (1974), no. 3, 521–528.
[14] I. Rudich and L.-M. Rousseau, *Deterministic factoring in time $N^{o(1)}$*, Hexagon preprint hexagon:2610.00600v1, published 29 October 2026, https://hexagonmath.org/2610.00600v1.
[15] I. Rudich and L.-M. Rousseau, *SubexponentialDeterministicFactoring*, Lean development, https://github.com/IsaacRudich/SubexponentialDeterministicFactoring, registered as Palomar entry PALOMAR-2026-10-28-000015 (version 1), Apache-2.0 licence.
[16] A. Schönhage and V. Strassen, *Schnelle Multiplikation großer Zahlen*, Computing **7** (1971), no. 3–4, 281–292.
[17] J. Shunia, *An improved subexponential bound for deterministic integer factorization*, Hexagon preprint hexagon:2610.00711v1, published 2 November 2026, https://hexagonmath.org/2610.00711v1.
[18] V. Strassen, *Einige Resultate über Berechnungskomplexität*, Jahresber. Deutsch. Math.-Verein. **78** (1976/77), no. 1, 1–8.
[19] V. Tulsyan, *Counting Hamiltonian cycles in sparse graphs in subexponential time, assuming the new #SAT algorithms*, Hexagon preprint hexagon:2610.00672v1, October 2026; source and Lean development at https://github.com/xennygrimmato/eth-hamiltonicity.
[20] V. Tulsyan, *eth-hamiltonicity: Lean 4 formalisation and research notes*, public repository https://github.com/xennygrimmato/eth-hamiltonicity, 2026; Lean sources as of commit `32509ed`.